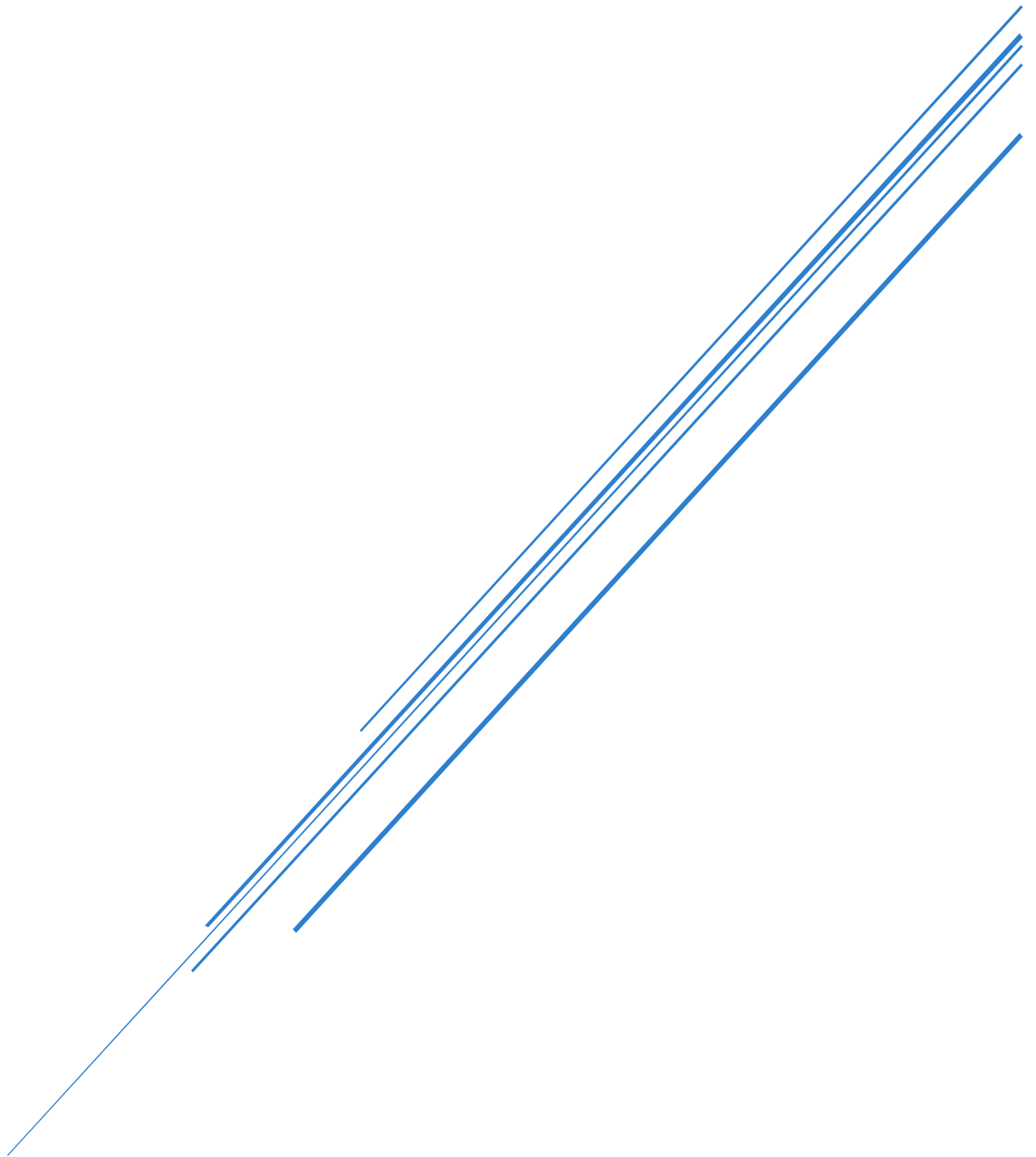


DATA PROCESSING STEPS



Machine Learning

Sr. No	Topics	Sub-topics
1	OUTLIERS HANDLING	
2	CATEGORICAL ENCODING	
3	FEATURE SCALING TECHNIQUES	
4	FEATURE SELECTIONS AND EXTRACTIONS TECHNIQUES	
5	HYPERPARAMETER TUNING METHODS	
6	BIAS AND VARIANCE	
7	DATA DRIFT VS MODEL DRIFT	
8	HANDLING IMBALANCED DATA FOR CLASSIFICATION	
9	EVALUATION METRICS FOR REGRESSION & CLASSIFICATION MODELS	
10	STEPS TO SOLVE ML PROBLEMS	

OUTLIERS HANDLING

- In machine learning, outliers are data points that deviate significantly from the general distribution of the dataset.
- They may occur due to errors in data collection, natural variation or rare events.
- While sometimes they contain useful insights like in fraud detection but in many cases, they negatively affect model accuracy and skew results making outlier detection a crucial preprocessing step.
- Impact on ML models: Can bias parameter estimation, increase variance and reduce model accuracy.
- Sources: Data entry errors, measurement noise, genuine rare events.
- Handling methods: Removal, transformation or robust algorithms less sensitive to outliers.
- Domain dependence: What is considered an outlier in one domain (e.g., finance) may be normal in another.

Outliers Handling Methods

1. Z-Score:

- The Z-Score method is a statistical technique that detects outliers based on how far a data point is from the mean, measured in terms of standard deviations.
- It assumes the data follows a normal distribution.
- A point with a very high or low Z-score (typically $|Z| > 3$) is flagged as an outlier because it lies in the extreme tails of the distribution.
- **Formula: $Z = (X - \mu) / \sigma$** , Where,
 - Z: The Z-score (standard score): It tells us how many standard deviations a data point is away from the mean.
 - x: The actual data value (the observation we are testing).
 - μ : The mean of the dataset (average of all data points).
 - σ : The standard deviation of the dataset (a measure of spread/variability).
- How it works: Compares distance of a point from the mean in units of standard deviation.
- Intuition: Outliers are “too far” from the centre of the bell curve.
- Pros: Simple, fast, works well with normally distributed data.
- Cons: Not reliable for skewed or non-normal distributions.

2. IQR Method (Interquartile Range):

- The IQR method is a robust statistical approach that identifies outliers by examining the spread of the middle 50% of the data.
- It calculates the Interquartile Range (IQR), which is the difference between the 75th percentile (Q3) and 25th percentile (Q1).
- Any value that falls **below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$** is considered an outlier.
- **$IQR = Q3 - Q1$**
 - Outliers Thresholds:

- Lower bound= $Q1 - 1.5 \times IQR$
- Upper bound= $Q3 + 1.5 \times IQR$
- Intuition: Values too far below or above the “box” in a boxplot are flagged.
- Pros: Robust to non-normal data, less influenced by extreme values.
- Cons: Doesn’t adapt well to very skewed distributions.

3. Isolation Forest:

- Isolation Forest is an outlier detection algorithm that works on one simple idea: “Outliers are rare and very different, so they are easier to isolate”.
- Core Intuition:
 - Imagine a group of points on a paper:
 - Normal points → crowded together
 - Outliers → far away, lonely
 - If you randomly draw lines to split the data:
 - Outliers get separated very quickly
 - Normal points take more splits
- Isolation Forest counts how fast a point gets isolated.
- Pros: Works well in high dimensions, efficient.
- Cons: Requires choosing contamination (expected outlier fraction).

4. Log Transformation:

- Log transformation reduces skewness and minimizes the effect of large outliers by compressing high values.
- Core Idea: Big numbers grow faster than small numbers — log slows them down
- Large values get compressed, small values change very little.
- Why Do We Need Log Transformation?
 - Real-world data is often:
 - Right-skewed
 - Has extreme values
 - Violates ML assumptions
- Log transformation helps to:
 - Reduce skewness
 - Reduce outlier influence
 - Improve linear relationships
- Stabilize variance
- Log transformation keeps the data but reduces dominance.
- When not to use it?
 - Data contains negative values
 - Tree-based models only (Random Forest, XGBoost)
 - Already normally distributed data

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.ensemble import IsolationForest
6 from sklearn.neighbors import LocalOutlierFactor
7 from scipy import stats

```

```

1 data = {
2     "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 50],
3     "Score": [35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 5]
4 }
5
6 df = pd.DataFrame(data)
7 df

```

	Study_Hours	Score
0	1	35
1	2	40
2	3	45
3	4	50
4	5	55
5	6	60
6	7	65
7	8	70
8	9	75
9	10	80
10	50	5

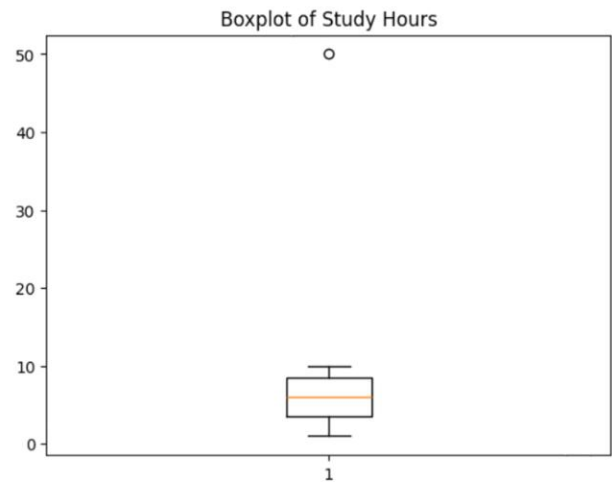
STEP 1: Visual Detection (EDA)

- The dot far above the box is an outlier
- Boxplot uses IQR internally

```

1 plt.boxplot(df["Study_Hours"])
2 plt.title("Boxplot of Study Hours")
3 plt.show()

```



STEP 2: IQR Method

```

1 Q1 = df["Study_Hours"].quantile(0.25)
2 Q3 = df["Study_Hours"].quantile(0.75)
3 IQR = Q3 - Q1
4
5 lower = Q1 - 1.5 * IQR
6 upper = Q3 + 1.5 * IQR
7
8 outliers_iqr = df[(df["Study_Hours"] < lower) | (df["Study_Hours"] > upper)]
9 outliers_iqr

```

	Study_Hours	Score
10	50	5

Removing Outliers (Trimming)

- Removes data completely
- Avoid if dataset is small

```

1 df_iqr_cleaned = df[(df["Study_Hours"] >= lower) & (df["Study_Hours"] <= upper)]
2 df_iqr_cleaned

```

	Study_Hours	Score
0	1	35
1	2	40
2	3	45
3	4	50
4	5	55

STEP 3: Z-Score Method

- Z-score > 3 → outlier
- Works best for normally distributed data

```

1 from scipy.stats import zscore
2
3 df["z_score"] = zscore(df["Study_Hours"])
4 df

```

	Study_Hours	Score	z_score
0	1	35	-0.653187
1	2	40	-0.576751
2	3	45	-0.500314
3	4	50	-0.423877
4	5	55	-0.347440
5	6	60	-0.271003
6	7	65	-0.194566
7	8	70	-0.118130
8	9	75	-0.041693
9	10	80	0.034744
10	50	5	3.092217

```

1 df[np.abs(df["z_score"]) > 3]

```

	Study_Hours	Score	z_score
10	50	5	3.092217

STEP 4: Capping (Winsorization)

- Best practice in ML pipelines
- Keeps dataset size same
- Reduces outlier influence

```
1 lower_cap = df["Study_Hours"].quantile(0.05)
2 upper_cap = df["Study_Hours"].quantile(0.95)
3
4 df["Study_Hours_Capped"] = np.clip(df["Study_Hours"], lower_cap, upper_cap)
5 df
```

	Study_Hours	Score	z_score	Study_Hours_Capped
0	1	35	-0.653187	1.5
1	2	40	-0.576751	2.0
2	3	45	-0.500314	3.0
3	4	50	-0.423877	4.0
4	5	55	-0.347440	5.0
5	6	60	-0.271003	6.0
6	7	65	-0.194566	7.0
7	8	70	-0.118130	8.0
8	9	75	-0.041693	9.0
9	10	80	0.034744	10.0
10	50	5	3.092217	30.0

STEP 5: Transformation (Log)

- Compresses large values
- Reduces skewness
- Improves linear models

STEP 6: Robust Scaling (ML-Friendly)

- Uses median & IQR
- Not affected by extreme values

```
1 from sklearn.preprocessing import RobustScaler
2
3 scaler = RobustScaler()
4 df["Study_Hours_Scaled"] = scaler.fit_transform(df[["Study_Hours"]])
5 df
```

	Study_Hours	Score	z_score	Study_Hours_Capped	Study_Hours_Scaled
0	1	35	-0.653187	1.5	-1.0
1	2	40	-0.576751	2.0	-0.8
2	3	45	-0.500314	3.0	-0.6
3	4	50	-0.423877	4.0	-0.4
4	5	55	-0.347440	5.0	-0.2
5	6	60	-0.271003	6.0	0.0
6	7	65	-0.194566	7.0	0.2
7	8	70	-0.118130	8.0	0.4
8	9	75	-0.041693	9.0	0.6
9	10	80	0.034744	10.0	0.8
10	50	5	3.092217	30.0	8.8

STEP 7: Isolation Forest (Advanced)

- -1 → outlier
- Detects multivariate anomalies
- Used in fraud & anomaly detection

```
1 from sklearn.ensemble import IsolationForest
2
3 iso = IsolationForest(contamination=0.1, random_state=42)
4 df["anomaly"] = iso.fit_predict(df[["Study_Hours", "Score"]])
5
6 df
```

	Study_Hours	Score	z_score	Study_Hours_Capped	Study_Hours_Scaled	anomaly
0	1	35	-0.653187	1.5	-1.0	1
1	2	40	-0.576751	2.0	-0.8	1
2	3	45	-0.500314	3.0	-0.6	1
3	4	50	-0.423877	4.0	-0.4	1
4	5	55	-0.347440	5.0	-0.2	1
5	6	60	-0.271003	6.0	0.0	1
6	7	65	-0.194566	7.0	0.2	1
7	8	70	-0.118130	8.0	0.4	1
8	9	75	-0.041693	9.0	0.6	1
9	10	80	0.034744	10.0	0.8	1
10	50	5	3.092217	30.0	8.8	-1

```
1 df[df["anomaly"] == -1]
```

Categorical Data Encoding Techniques

Types of Categorical Data

1. Nominal Data:

- Nominal data consists of categories without any inherent order or ranking. These are simple labels used to classify data.
- Example: 'Red', 'Blue', 'Green' (Car Colour).
- Encoding Options: One-Hot Encoding or Label Encoding, depending on the model's needs.

2. Ordinal Data:

- Ordinal data includes categories with a defined order or ranking, where the relationship between values is important.
- Example: 'Low', 'Medium', 'High' (Car Engine Power).
- Encoding Options: Ordinal Encoding.

1. Label Encoding

- Label Encoding assigns each category a unique integer. It is simple and memory-efficient but may unintentionally imply an order among categories when none exists.
- Used in tree-based models like Decision Trees or XGBoost.
- Pros: Simple and memory-efficient.
- Cons: Introduces implicit order which may be misinterpreted by non-tree models when used with nominal data.

```
1 import pandas as pd
2 import numpy as np
3
4 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```



```
1 data = ['Red', 'Green', 'Blue', 'Red']
2
3 le = LabelEncoder()
4 encoded_data = le.fit_transform(data)
5 print(f"Encoded Data: {encoded_data}")
```

Encoded Data: [2 1 0 2]

2. One-Hot Encoding

- One-Hot Encoding converts categories into binary columns with each column representing one category. It prevents false ordering but can lead to high dimensionality if there are many unique values.

- Used in linear models, logistic regression and neural networks.
- Pros: Does not assume order; widely supported.
- Cons: Can cause high dimensionality and sparse data when feature has many categories.

```
1 data = ['Red', 'Blue', 'Green', 'Red']
2
3 df = pd.DataFrame(data, columns=['Color'])
4 one_hot_encoded = pd.get_dummies(df['Color'])
5
6 print(one_hot_encoded)
```

	Blue	Green	Red
0	False	False	True
1	True	False	False
2	False	True	False
3	False	False	True

```
1 one_encode = OneHotEncoder()
2 encoded_col = one_encode.fit_transform(df)
3
4 encoded_df = pd.DataFrame(
5     encoded_col.toarray(),
6     columns=one_encode.get_feature_names_out(['Color'])
7 )
8
9 print(encoded_df)
```

	Color_Blue	Color_Green	Color_Red
0	0.0	0.0	1.0
1	1.0	0.0	0.0
2	0.0	1.0	0.0
3	0.0	0.0	1.0

3. Ordinal Encoding:

- Ordinal Encoding maps categories to integers while preserving their natural order. This works well for ordered data like ratings but is not suitable for nominal variables.
- Used for ordered features like ratings or education levels.
- Pros: Maintains order; reduces dimensionality.
- Cons: Not suitable for nominal categories.

```
1 from sklearn.preprocessing import OrdinalEncoder
2 data = [['Low'], ['Medium'], ['High'], ['Medium'], ['Low']]
3
4 encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
5 encoded_data = encoder.fit_transform(data)
6
7 print(f"Encoded Ordinal Data: {encoded_data}")
```

```
Encoded Ordinal Data: [[0.]
[1.]
[2.]
[1.]
[0.]]
```

4. Target Encoding

- Target Encoding also known as Mean Encoding is a technique where each category in a feature is replaced by the mean of the target variable for that category.
- Useful for high-cardinality features like ZIP codes or product IDs.
- Pros: Captures relationship to target variable.
- Cons: Risk of overfitting, also must apply smoothing/statistical techniques.

```
1 import pandas as pd
2 import category_encoders as ce
3
4 df = pd.DataFrame(
5     {'City': ['London', 'Paris', 'London', 'Berlin'], 'Target': [1, 0, 1, 0]}
6 )
7
8 encoder = ce.TargetEncoder(cols=['City'])
9 df_tgt = encoder.fit_transform(df['City'], df['Target'])
10
11 print(f"Encoded Target Data:\n{df_tgt}")
```

Encoded Target Data:

	City
0	0.570926
1	0.434946
2	0.570926
3	0.434946

5. Binary Encoding

- Binary encoding represents categories as binary codes and splits them across multiple columns. It is efficient for high-cardinality data but slightly more complex to implement.
- Applied in high-cardinality text/NLP tasks to save memory.
- Pros: Reduces dimensionality, more memory-efficient than one-hot encoding.
- Cons: Slightly more complex; requires careful handling of missing values.

```
1 import category_encoders as ce
2 data = ['Red', 'Green', 'Blue', 'Red']
3 encoder = ce.BinaryEncoder(cols=['Color'])
4 encoded_data = encoder.fit_transform(pd.DataFrame(data, columns=['Color']))
5 print(encoded_data)
```

	Color_0	Color_1
0	0	1
1	1	0
2	1	1
3	0	1

6. Frequency Encoding

- Frequency Encoding assigns categories values based on how often they occur in the dataset. It is simple and compact but can introduce data leakage if applied improperly.

- Effective in retail, e-commerce or clickstream data for popularity trends.
- Pros: Low computational and storage requirements.
- Cons: Can introduce data leakage if not handled properly.

```
1 data = ['Red', 'Green', 'Blue', 'Red', 'Red']
2 series_data = pd.Series(data)
3 frequency_encoding = series_data.value_counts()
4
5 encoded_data = [frequency_encoding[x] for x in data]
6 print("Encoded Data:", encoded_data)
```

```
Encoded Data: [np.int64(3), np.int64(1), np.int64(1), np.int64(3), np.int64(3)]
```

Feature Scaling Techniques

- In machine learning, **feature scaling** is a critical preprocessing step that standardizes or normalizes data so that different features contribute equally to the learning process.
- Many machine learning algorithms are sensitive to the magnitude of the features, which means that unscaled features can lead to poor model performance, slower convergence, or even failure to learn meaningful patterns. However, not all algorithms require feature scaling.

Why is Feature Scaling Important?

- Feature scaling ensures that all the input variables (features) contribute equally to the model's learning process.
- When features have different units or magnitudes (e.g., age in years vs. income in dollars), models may become biased toward the features with larger values, ignoring those with smaller values, or they may converge slowly.

1. Absolute Maximum Scaling

- Absolute Maximum Scaling rescales each feature by dividing all values by the maximum absolute value of that feature.
- This ensures the feature values fall within the range of -1 to 1. While simple and useful in some contexts, it is highly sensitive to outliers which can skew the max absolute value and negatively impact scaling quality.
- $X_{\text{scaled}} = \frac{x}{\max(|x|)}$
- Scales values between -1 and 1.
- Sensitive to outliers, making it less suitable for noisy datasets.

```
import pandas as pd
import numpy as np

df = pd.read_csv('Housing.csv')

df = df.select_dtypes(include=np.number)
df.head()
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	13300000	7420	4	2	3	2
1	12250000	8960	4	4	4	3
2	12250000	9960	3	2	2	2
3	12215000	7500	4	2	2	3
4	11410000	7420	4	1	2	2

Dataset

```
max_abs = np.max(np.abs(df), axis=0)
scaled_df = df / max_abs
scaled_df.head()
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	1.000000	0.458025	0.666667	0.50	0.75	0.666667
1	0.921053	0.553086	0.666667	1.00	1.00	1.000000
2	0.921053	0.614815	0.500000	0.50	0.50	0.666667
3	0.918421	0.462963	0.666667	0.50	0.50	1.000000
4	0.857895	0.458025	0.666667	0.25	0.50	0.666667

Absolute Maximum Scaling

2. Min-Max Scaling

- Min-Max Scaling transforms features by subtracting the minimum value and dividing by the difference between the maximum and minimum values.
- This method maps feature values to a specified range, commonly 0 to 1, preserving the original distribution shape but is still affected by outliers due to reliance on extreme values.
- Scales feature to range.
- Sensitive to outliers because min and max can be skewed.

$$X_{\text{scaled}} = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

scaled_df.head()
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	1.000000	0.396564	0.6	0.333333	0.666667	0.666667
1	0.909091	0.502405	0.6	1.000000	1.000000	1.000000
2	0.909091	0.571134	0.4	0.333333	0.333333	0.666667
3	0.906061	0.402062	0.6	0.333333	0.333333	1.000000
4	0.836364	0.396564	0.6	0.000000	0.333333	0.666667

Min-Max Scaling

3. Normalization (Vector Normalization)

- **Vector Normalization** scales an entire **row (data point)** so that its **magnitude (length)** becomes **1**.
It is also called **unit norm scaling**.
- This focuses on the direction of data points rather than magnitude making it useful in algorithms where angle or cosine similarity is relevant, such as text classification or clustering.

$$X_{\text{scaled}} = \frac{X_i}{\|X\|}$$

```
from sklearn.preprocessing import Normalizer

scaler = Normalizer()
scaled_data = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

scaled_df.head()
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	1.0	0.000558	3.007518e-07	1.503759e-07	2.255639e-07	1.503759e-07
1	1.0	0.000731	3.265305e-07	3.265305e-07	3.265305e-07	2.448979e-07
2	1.0	0.000813	2.448979e-07	1.632653e-07	1.632653e-07	1.632653e-07
3	1.0	0.000614	3.274662e-07	1.637331e-07	1.637331e-07	2.455996e-07
4	1.0	0.000650	3.505696e-07	8.764240e-08	1.752848e-07	1.752848e-07

Normalization

4. Standardization (Z-Score Scaling)

- **Standardization** rescales features so that they have **mean = 0** and **standard deviation = 1**.

$$X_{\text{scaled}} = \frac{X_i - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_data,
                        columns=df.columns)

print(scaled_df.head())
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	4.566365	1.046726	1.403419	1.421812	1.378217	1.517692
1	4.004484	1.757010	1.403419	5.405809	2.532024	2.679409
2	4.004484	2.218232	0.047278	1.421812	0.224410	1.517692
3	3.985755	1.083624	1.403419	1.421812	0.224410	2.679409
4	3.554979	1.046726	1.403419	-0.570187	0.224410	1.517692

Standardization

5. Robust Scaling

- Robust Scaling uses the median and interquartile range (IQR) instead of the mean and standard deviation making the transformation robust to outliers and skewed distributions.
- It is highly suitable when the dataset contains extreme values or noise.

$$X_{\text{scaled}} = \frac{X_i - X_{\text{median}}}{IQR}$$

- Reduces influence of outliers by centering on median
- Scales based on IQR, which captures middle 50% spread

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
scaled_data = scaler.fit_transform(df)
scaled_df = pd.DataFrame(scaled_data,
                          columns=df.columns)

print(scaled_df.head())
```

Output:

	price	area	bedrooms	bathrooms	stories	parking
0	3.878788	1.021739	1.0	1.0	1.0	2.0
1	3.424242	1.579710	1.0	3.0	2.0	3.0
2	3.424242	1.942029	0.0	1.0	0.0	2.0
3	3.409091	1.050725	1.0	1.0	0.0	3.0
4	3.060606	1.021739	1.0	0.0	0.0	2.0

Robust Scaling

Data Sampling Methods

Types of Data Sampling

- You download a relatively big dataset and are excited to get started with analysing it and building your machine learning model. And snap – your machine gives an “out of memory” error while trying to load the dataset. In such cases, employing efficient data loading techniques and utilizing types of sampling can help manage memory constraints and facilitate analysis.
- It’s happened to the best of us. It’s one of the biggest hurdles we face in data science – dealing with massive amounts of data on computationally limited machines.
- Sampling is a great way to pick up a subset of the data and analyse that.

What is Sampling?

- Data sampling is the process of selecting a subset of data (sample) from a larger dataset (population) to analyse, model, or draw conclusions from, without using the entire dataset.

Why do we need data sampling?

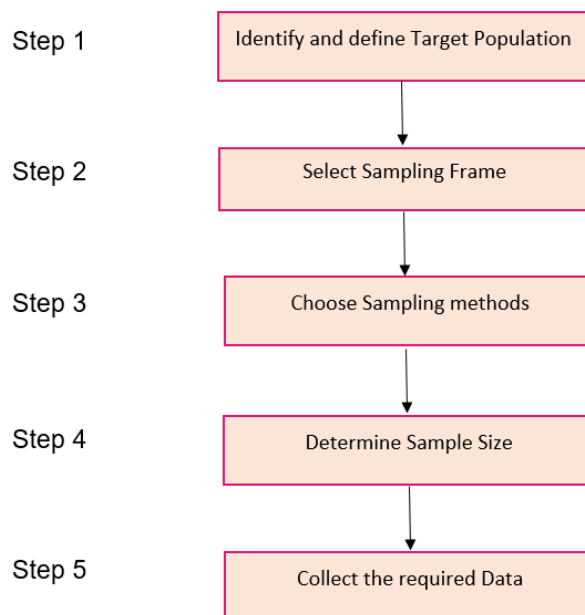
- **Large datasets** are expensive to process
- Saves **time and computational cost**
- Makes **model training faster**
- Useful when full data is **not available or too large**
- Helps in **quick experimentation and prototyping**

Steps involved in Sampling

Let’s take an interesting case study and apply these steps to perform sampling. We recently conducted General Elections in India a few months back. You must have seen the public opinion polls every news channel was running at the time:

Step 1:

- The first stage in the sampling process is to clearly define the target population.
- So, to carry out opinion polls, polling agencies consider only the people who are above 18 years of age and are eligible to vote in the population.



Step 2:

- **Sampling Frame** – It is a list of items or people forming a population from which the sample is taken.
- So, the sampling frame would be the list of all the people whose names appear on the voter list of a constituency.

Step 3:

- Generally, probability sampling methods use every vote's equal value, allowing any person to join the sample regardless of their caste, community, or religion. Researchers take different samples from various regions across the country.

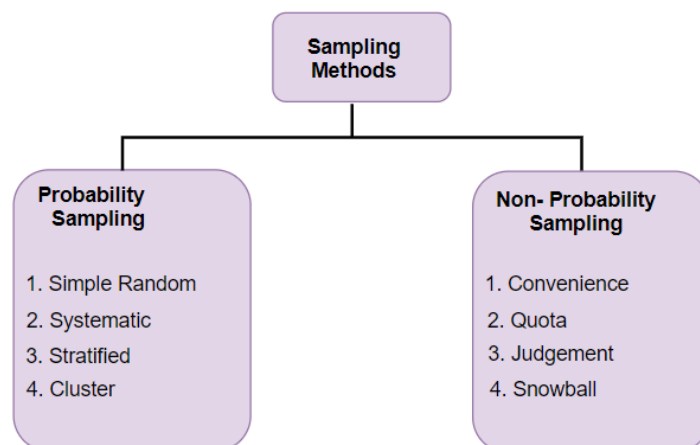
Step 4:

- **Sample Size** – The number of individuals or items to include in a sample must be sufficient to make inferences about the population with the desired level of accuracy and precision.
- Larger the sample size, more accurate our inference about the population would be.
- For the polls, agencies strive to include as many people as possible from diverse backgrounds in the sample, as this inclusion helps predict the number of seats a political party can win.

Step 5:

- Once the target population, sampling frame, sampling technique, and sample size have been established, the next step is to **collect data from the sample**.
- In opinion polls, agencies generally put questions to the people, like which political party are they going to vote for or has the previous party done any work, etc.
- Based on the answers, agencies try to interpret who the people of a constituency are going to vote for and approximately how many seats is a political party going to win.

Different Types of Sampling



- **Probability Sampling:** In probability sampling, every element of the population has an equal chance of being selected. Probability sampling gives us the best chance to create a sample that is truly representative of the population.
- **Non-Probability Sampling:** In non-probability sampling, all elements do not have an equal chance of being selected. Consequently, there is a significant risk of ending up with a non-representative sample which does not produce generalizable results.

1. Random Sampling

Each data point has an equal chance of being selected.

- Example: Randomly selecting 1,000 rows from a dataset

✓ Simple and unbiased

✗ May miss important subgroups in small samples.

2. Stratified Sampling

Data is divided into groups (**strata**) and samples are taken from each group.

- Example: Sampling students from each grade proportionally

✓ Maintains class distribution

✓ Very useful in **imbalanced datasets**

3. Systematic Sampling

Select every k^{th} data point.

- Example: Select every 10th record

✓ Easy to implement

✗ Risk if data has patterns

- Suppose, we began with person number 3, and we want a sample size of 5. So, the next individual that we will select would be at an interval of $(20/5) = 4$ from the 3rd person, i.e. 7 ($3+4$), and so on.

4. Cluster Sampling

Entire groups (clusters) are randomly selected.

- Example: Select 5 schools and include all students from them

✓ Cost-effective

✗ Less accurate than stratified sampling

5. Convenience Sampling

Sampling based on ease of access.

- Example: First 1,000 rows

✗ Biased

✗ Not recommended for ML or statistics

6. Quota Sampling

Quota sampling is a **non-probability sampling** method where the researcher:

- Divides the population into **subgroups (quotas)**
- Collects data until each quota is filled
- Selection within each group is **not random**

Example

Suppose you are surveying **100 people**:

- 60% males → collect **60 males**
- 40% females → collect **40 females**

You stop once each quota is completed.

Key points

✓ Ensures representation of groups

✗ Selection is biased

✗ No randomness

Where used

- Market research

- Opinion surveys
- When population proportions are known

7. Judgment Sampling (Purposive Sampling)

Judgment sampling selects samples **based on the researcher's knowledge and judgment** about who will give the best information.

Example

- Interview **experienced data scientists** to understand ML trends
- Select **senior doctors** for medical research

The researcher *purposely* chooses experts.

Key points

- ✓ High-quality, relevant data
- ✓ Useful for expert opinions
- ✗ Highly subjective
- ✗ Cannot generalize results

Where used

- Qualitative research
- Case studies
- Expert-based analysis

8. Snowball Sampling

Snowball sampling is used when the population is **hard to find**.

- One participant refers others
- Sample grows like a snowball

Example

- Drug users
- Rare disease patients
- Underground communities

One participant → refers 2 → they refer more → sample expands.

Key points

- ✓ Effective for hidden populations
- ✓ Low cost

✗ Highly biased

✗ Over-representation of connected groups

Where used

- Social research
- Medical & psychological studies
- Sensitive populations

In ML, sampling is used for:

- **Train-test split**
- **Handling imbalanced data**
 - Oversampling (SMOTE)
 - Under sampling
- **Cross-validation**
- **Mini-batch training**

Hyperparameter Tuning Methods

- Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameter. These are typically set before the actual training process begins and control aspects of the learning process itself.
- Effective tuning helps the model learn better patterns, avoid overfitting or underfitting and achieve higher accuracy on unseen data.

What is Cross Validation?

- Before diving into hyperparameter tuning, it is essential to evaluate the model properly. Instead of relying on a simple train-test split, we use K-Fold Cross-Validation. This technique divides the dataset into k subsets (folds). The model is trained on k-1 folds and tested on the remaining fold. This process repeats k times, with each fold serving as the test set once. Finally, the average score across all iterations provides a more reliable performance estimate, reducing the risk of overfitting.
- For imbalanced datasets, Stratified K-Fold Cross-Validation ensures that each fold maintains the same class distribution as the original dataset, leading to more consistent results.

Methods for Hyperparameter Tuning:

1. Manual Hyperparameter Tuning:

- Manual tuning involves trial and error, where a data scientist adjusts hyperparameters and observes model performance.
- While this is easy for small models, it becomes impractical as the number of hyperparameters increases.

2. GridSearchCV:

- GridSearchCV is a brute-force technique for hyperparameter tuning. It trains the model using all possible combinations of specified hyperparameter values to find the best-performing setup. It is slow and uses a lot of computer power which makes it hard to use with big datasets or many settings. It works using below steps:
 - Create a grid of potential values for each hyperparameter.
 - Train the model for every combination in the grid.
 - Evaluate each model using cross-validation.
 - Select the combination that gives the highest score.
- For example, if we want to tune two hyperparameters C and Alpha for a Logistic Regression Classifier model with the following sets of values:

C	0.5	0.701	0.703	0.697	0.696
	0.4	0.699	0.702	0.698	0.702
	0.3	0.721	0.726	0.713	0.703
	0.2	0.706	0.705	0.704	0.701
	0.1	0.698	0.692	0.688	0.675
		0.1	0.2	0.3	0.4
		Alpha			

- The grid search technique will construct multiple versions of the model with all possible combinations of C and Alpha, resulting in a total of $5 * 4 = 20$ different models. The best-performing combination is then chosen.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define hyperparameters
param_grid = {
    'n_estimators': [10, 50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

# Initialize model
rf = RandomForestClassifier()

# Perform Grid Search
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Best parameters and score
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

Pro: Finds the best combination exhaustively.

Con: Computationally expensive for large datasets and many hyperparameters.

3. RandomizedSearchCV:

- As the name suggests RandomizedSearchCV picks random combinations of hyperparameters from the given ranges instead of checking every single combination like GridSearchCV.
- In each iteration it tries a new random combination of hyperparameter values.
- It records the model's performance for each combination.
- After several attempts it selects the best-performing set.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

# Define hyperparameters with a range of values
param_dist = {
    'n_estimators': randint(10, 200),
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': randint(2, 11)
}

# Perform Randomized Search
random_search = RandomizedSearchCV(rf, param_dist, n_iter=10, cv=5, scoring='accuracy')
random_search.fit(X_train, y_train)

print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)
```

Pro: Faster than Grid Search, good for large datasets.

Con: Might miss the optimal combination.

4. Bayesian Optimization:

- Grid Search and Random Search can be inefficient because they blindly try many hyperparameter combinations, even if some are clearly not useful.
- Bayesian Optimization takes a smarter approach. It treats hyperparameter tuning like a mathematical optimization problem and learns from past results to decide what to try next.
- Build a probabilistic model (surrogate function) that predicts performance based on hyperparameters.
- Update this model after each evaluation.
- Use the model to choose the next best set to try.
- Repeat until the optimal combination is found. The surrogate function models:

$$P(\text{score}(y) \mid \text{hyperparameters}(x))$$

- Here the surrogate function models the relationship between hyperparameters x and the score y . By updating this model iteratively with each new evaluation Bayesian optimization makes more informed decisions. Common surrogate models used in Bayesian optimization include:
 - Gaussian Processes
 - Random Forest Regression
 - Tree-structured Parzen Estimators (TPE)

```
from skopt import BayesSearchCV
from skopt.space import Integer, Categorical

# Define hyperparameters with a range of values
param_space = {
    'n_estimators': Integer(10, 200),
    'max_depth': Categorical([None, 10, 20, 30]),
    'min_samples_split': Integer(2, 10)
}

# Perform Randomized Search
bayes_search = BayesSearchCV(rf, param_space, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
bayes_search.fit(X_train, y_train)

print("Best Parameters:", bayes_search.best_params_)
print("Best Score:", bayes_search.best_score_)

Best Parameters: OrderedDict({'max_depth': 30, 'min_samples_split': 9, 'n_estimators': 68})
Best Score: 0.9583333333333334
```

Advantages of Hyperparameter tuning:

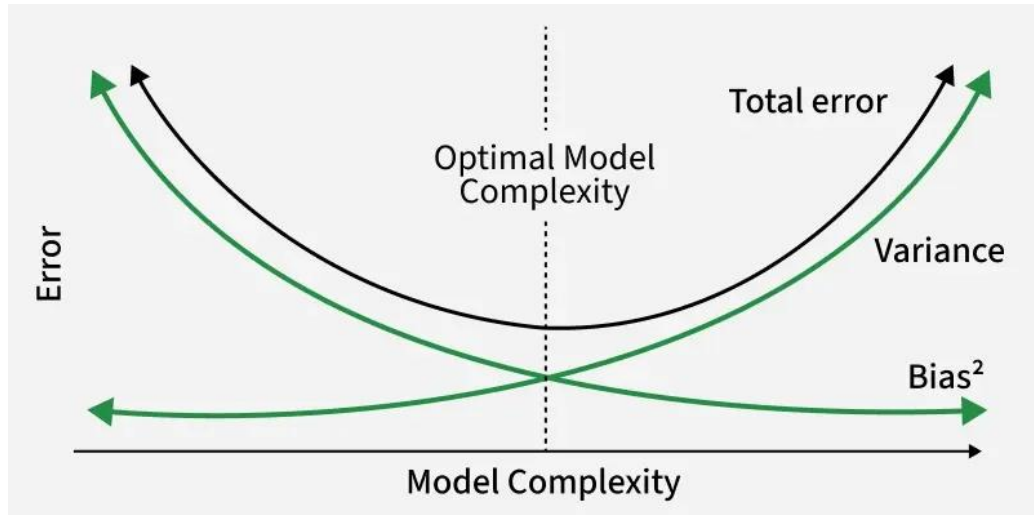
- **Improved Model Performance:** Finding the optimal combination of hyperparameters can significantly boost model accuracy and robustness.
- **Reduced Overfitting and Underfitting:** Tuning helps to prevent both overfitting and underfitting resulting in a well-balanced model.
- **Enhanced Model Generalizability:** By selecting hyperparameters that optimize performance on validation data the model is more likely to generalize well to unseen data.
- **Optimized Resource Utilization:** With careful tuning resources such as computation time and memory can be used more efficiently avoiding unnecessary work.
- **Improved Model Interpretability:** Properly tuned hyperparameters can make the model simpler and easier to interpret.

Challenges:

- **Dealing with High-Dimensional Hyperparameter Spaces:** The larger the hyperparameter space the more combinations need to be explored. This makes the search process computationally expensive and time-consuming especially for complex models with many hyperparameters.
- **Incorporating Domain Knowledge:** It can help guide the hyperparameter search, narrowing down the search space and making the process more efficient. Using insights from the problem context can improve both the efficiency and effectiveness of tuning.
- **Developing Adaptive Hyperparameter Tuning Methods:** Dynamic adjustment of hyperparameters during training such as learning rate schedules or early stopping can lead to better model performance.

Bias and Variance

- Bias and Variance are two fundamental concepts that help explain a model's prediction errors in machine learning.
- Bias refers to the error caused by oversimplifying a model.
- Variance refers to the error from making the model too sensitive to training data.



Bias

Bias is the error that occurs when a model is too simple to capture the true patterns in the data.

1. **High bias:** The model oversimplifies, misses patterns and underfits the data.
2. **Low bias:** The model captures patterns well and is closer to the true values.

Example: A neural network with too few layers or neurons fails to capture complex patterns, producing consistently inaccurate outputs. This is called underfitting.

How to Reduce Bias?

Some methods to lower bias in models are:

1. **Use More Complex Models:** Use models capable of capturing non-linear relationships such as neural networks or ensemble methods.
2. **Add Relevant Features:** Include additional informative features in the training data to give the model for capturing underlying patterns.
3. **Adjust Regularization Strength:** Reduce regularization to allow the model more flexibility in fitting the data.

Variance

Variance arises when a model becomes too sensitive to training data and it captures noises in data too. It fails to give prediction on unseen new data.

1. **High variance:** The model is too sensitive to small changes and may overfit.
2. **Low variance:** The model is more stable but might miss some patterns.

Example: A deep decision tree that memorizes the training data perfectly but performs poorly on new data shows high variance, this is known as overfitting.

How to Reduce Variance?

Some methods to lower variance are:

1. **Simplify the Model:** Use a simpler model or prune overly deep decision trees to avoid overfitting.
2. **Increase Training Data:** Collect more data to stabilize learning and make the model generalize better.
3. **Apply Regularization:** Use L1 or L2 regularization to constrain model complexity and prevent overfitting.
4. **Use Ensemble Methods:** Implement techniques like bagging or random forests to combine multiple models and balance bias–variance trade-offs.

Applications

Some of the applications of bias and variance analysis are:

1. **Model Selection:** Helps determine whether a simple or complex model is best suited for the task ensuring good generalization.
2. **Hyperparameter Tuning:** Guides fine tuning parameters such as learning rate, regularization strength or tree depth to reduce errors.
3. **Model Evaluation:** Assists in identifying underfitting or overfitting by comparing training and test performance.
4. **Error Analysis:** Helps pinpoint the main causes of prediction errors and refine model strategies accordingly.
5. **Ensemble Learning:** Balances bias and variance effectively by combining multiple models to enhance stability and accuracy.

Advantages

Some of the advantages of understanding bias and variance are:

1. **Improves Model Accuracy:** Enables building models that perform consistently well on unseen data.
2. **Supports Efficient Training:** Saves computational resources by avoiding unnecessarily complex or overfitted models.

3. **Enhances Interpretability:** Makes it easier to understand and explain the reasons behind model errors.
4. **Guides Model Complexity:** Helps find the optimal level of model complexity for different data sizes and problems.

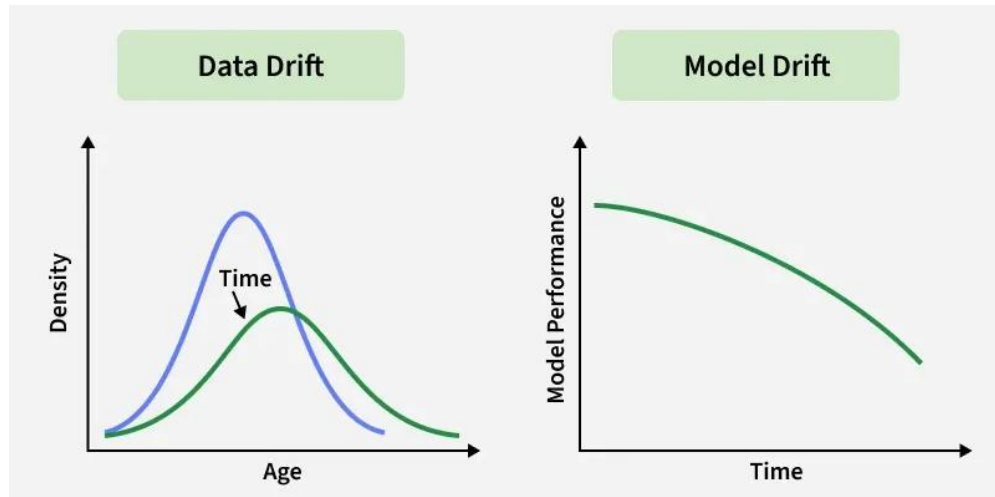
Limitations

Some of the limitations of bias and variance concepts are:

1. **Difficult to Quantify Precisely:** Measuring exact bias and variance in modern complex models can be challenging.
2. **Highly Data Dependent:** Model behaviour may vary significantly across datasets with different characteristics.
3. **Unpredictable in Deep Learning:** Deep neural networks can display unexpected bias-variance dynamics due to non-convex optimization.
4. **Trade-off Challenge:** Minimizing one often increases the other requiring careful experimentation and balance.

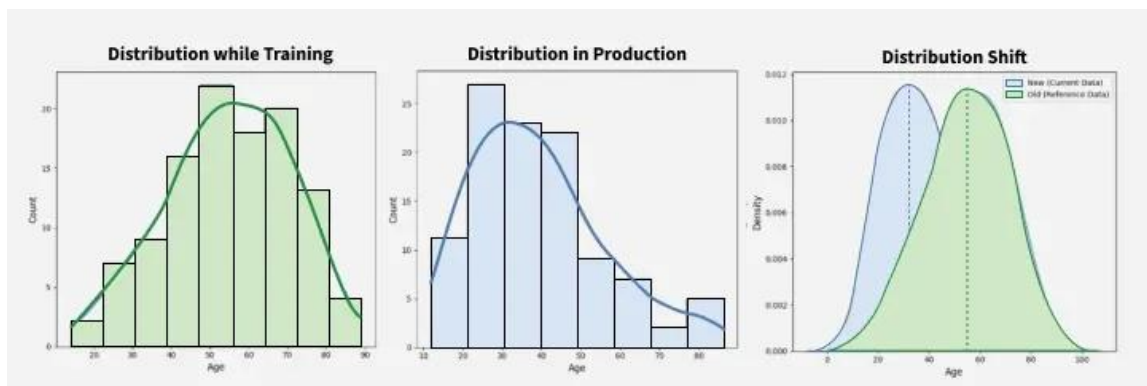
Data Drift vs Model Drift

Machine learning systems often degrade in performance after deployment due to evolving data patterns and shifts in real-world behaviour. Two common issues responsible for this decline are Data Drift and Model Drift. It impacts prediction accuracy, reliability and business decisions over time.



Data Drift

- Data drift is a situation where the statistical properties of the input data to a machine learning model change over time.
- When data drift occurs, the relationships between the features and the target variable are no longer valid. This can lead to poor model performance, inaccurate predictions and even failures.



Why do models drift

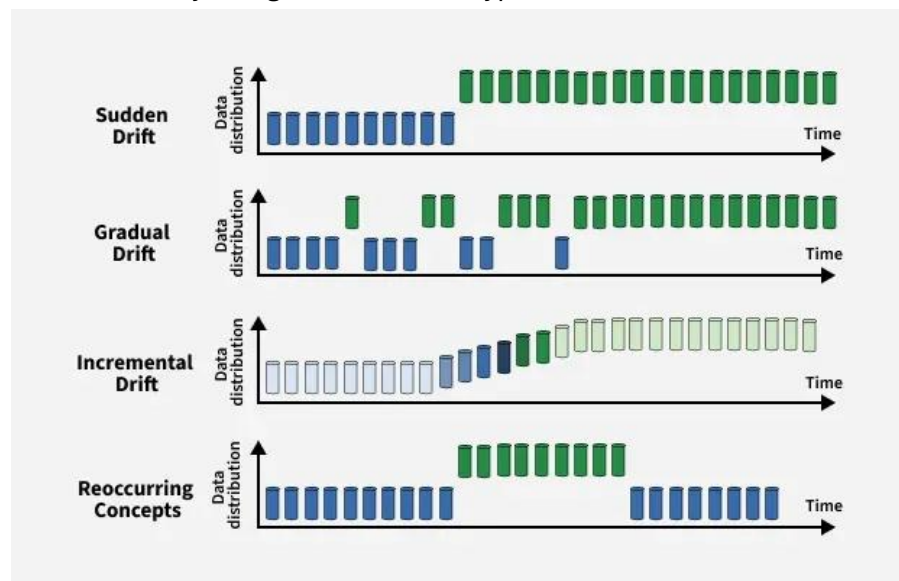
- Machine learning models can experience drift over time for several reasons. One common cause is when the data used to train the model becomes outdated or no longer reflects current conditions.
- Another reason for model drift is that some models are not designed to adapt to changes in the data.
- While certain machine learning models are better equipped to handle these changes, no model is entirely immune to drift.

- For example, suppose a machine learning model designed to predict a company's stock price using historical data. If the model is trained on data from a stable market, it may perform well initially. However, if the market becomes more volatile, the model may struggle to make accurate predictions because the statistical characteristics of the data have changed.

Types of Data Drift

1. Concept Drift

- Concept Drift refers to the change in the statistical relationship between input features (X) and the target variable (Y) over time. This implies that the patterns learned by a machine learning model at one point may no longer hold true in the future, which affects the model's performance and generalization ability.
- Concept drift can be broadly categorized into four types:



1. **Sudden Drift:** A rapid change in data distribution, where the new concept entirely replaces the old one. This often leads to an immediate drop in model accuracy if not handled properly.
2. **Gradual Drift:** The data distribution changes slowly over time, allowing old and new concepts to coexist for a while. The model may still perform reasonably well during this transition, but performance degrades if adaptation is delayed.
3. **Incremental Drift:** The shift in concept occurs through a sequence of small, incremental changes. Each step introduces a slight deviation, eventually leading to a completely different concept. This type of drift can be subtle and difficult to detect.
4. **Recurring or Seasonal Drift:** Previously observed concepts reappear after a period. For example, seasonal patterns in retail sales or cyclic user behaviour may cause certain data distributions to reoccur, requiring models to retain memory of past states.

2. Covariate Drift

- Covariate Drift occurs when the distribution of the input features changes over time, while the conditional relationship between the inputs (X) and the target output (Y) remains unchanged.
- In simple words, the way inputs are distributed shifts, but the underlying mapping from X to Y stays consistent.

- This can still impact model performance if the model is exposed to input patterns it has not encountered during training.

Algorithms for Detecting Data Drift

1. Kolmogorov-Smirnov (K-S) Test

The Kolmogorov-Smirnov (K-S) test is a statistical test used to determine whether two datasets originate from the same distribution. It does not assume any specific distribution shape which makes it suitable for a wide range of applications. It is often applied in situations where we want to test whether a sample of data is drawn from a specific population or to compare two different samples to assess whether they come from the same distribution.

Mechanism:

- The test works by comparing the **cumulative distribution functions (CDFs)** of two datasets. It looks at how the values in each dataset are spread out and adds them up step by step.
- The default assumption of the test is that both datasets come from the same distribution. If this assumption is rejected, it means there is likely a difference in the distributions indicating data drift.

2. Population Stability Index (PSI)

The Population Stability Index (PSI) is a statistical measure used to assess how much the distribution of a variable has changed between two datasets. It is primarily used for tracking changes in categorical or binned continuous variables over time. Though it was originally developed for monitoring credit scorecard stability in the financial domain, PSI is now widely used for evaluating shifts in distributions of both features and target variables in machine learning pipelines.

Mechanism:

- The PSI compares the proportion of observations from each bin of the variable's distribution. Checked in both the training dataset and the incoming or test dataset.
- The difference in these proportions across all bins is aggregated into a single PSI score.
 - **$PSI < 0.1$** : No significant change in the distribution.
 - **$0.1 \leq PSI < 0.25$** : Moderate change, may warrant closer monitoring.
 - **$PSI \geq 0.25$** : Significant drift, model retraining or recalibration likely needed.

3. Page-Hinkley Method

The Page-Hinkley method is a sequential technique to detect changes in the mean of a data stream over time. This method is commonly used for monitoring time series data or streaming data where early detection of change is essential. It is frequently applied to monitor model performance and detect shifts that may not be obvious at first glance.

Mechanism:

- A threshold value and a decision function are defined. The threshold determines how large a change in the mean must be for consideration. The decision function evaluates a statistically meaningful shift has occurred at each time step.

- Then we compute the cumulative difference between observed values and their mean.
- Monitor the cumulative difference to detect deviations that exceed the defined threshold.

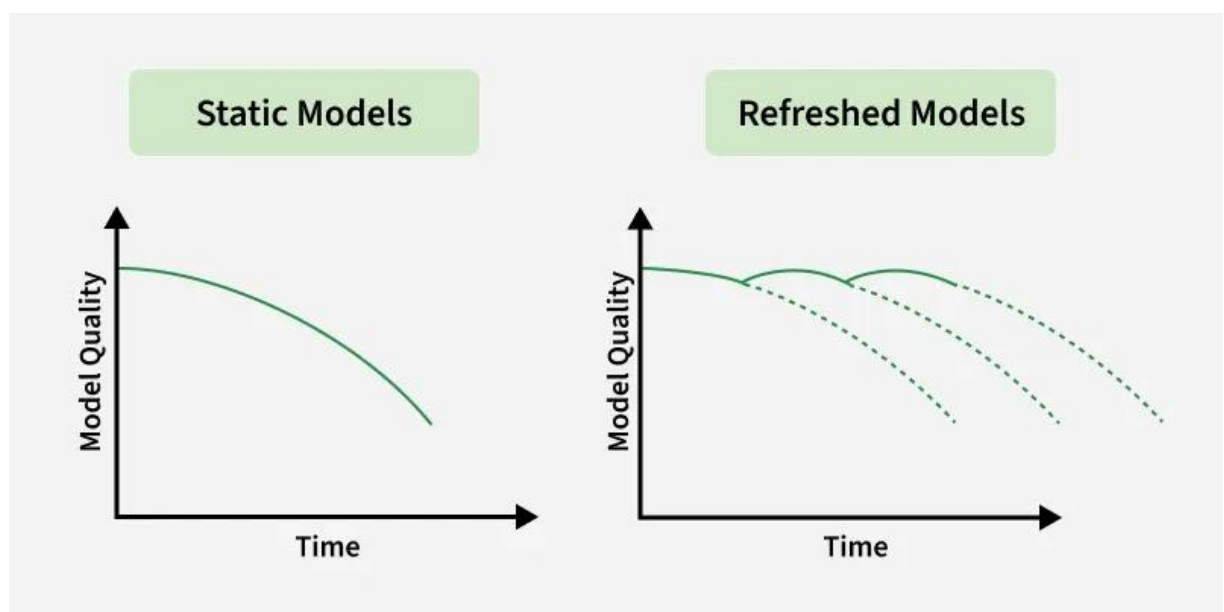
Managing Data Drift

Once data drift is detected, it is important to take action to ensure the model remains accurate and reliable. There are several methods to handling data drift effectively:

- **Retraining the Model:** It allows the model to learn new patterns and relationships that may have evolved over time. This strategy is best used when enough labelled data is available and computational resources allow for full retraining without major constraints.
- **Updating the Model with New Data:** When full retraining is impractical, models can be updated incrementally using new data. Transfer learning allows tuning of pre-trained models with recent data. This approach is ideal for real-time systems or systems where resources are limited.
- **Feature Engineering Adjustments:** Data drift can reduce the relevance of certain features, which makes feature engineering a critical adjustment strategy. New features that better capture current trends should be introduced and irrelevant ones should be removed. This keeps the model's input aligned with the evolving data landscape.
- **Ensemble Learning:** Using an ensemble of models trained on different time periods helps neutralize the effects of data drift. Methods like bagging, boosting or stacking combine multiple predictions. This enhances model stability even when individual models perform poorly due to data drift.

Model Drift

- Model drift refers to the gradual degradation in a deployed model's performance due to changes in real world data or relationships between input features and the target.
- As environments evolve, predictions become less accurate, requiring continuous monitoring and updates.



Reasons for Model Drift

Some common reasons behind model drift are:

- Real-world data distribution changes over time.
- Feature relationships shift due to behaviour or policy changes.
- External factors affect trends not seen in training data.
- Seasonal events temporarily disrupt model patterns.
- New patterns emerge that the model wasn't trained on.

Preventing Model Drift

Some preventive strategies are:

1. **Frequent Retraining:** Regularly retrain models with the latest data to ensure updated decision boundaries and improved generalization.
2. **Feature Validation:** Continuously validate whether input features remain relevant and stable, remove or adjust those losing predictive power.
3. **Feedback Loops:** Collect real-world outcomes, compare them with predictions and feed corrections back into the model pipeline to refine performance.
4. **Data Quality Checks:** Monitor missing values, noise and anomalies to ensure consistent input distribution.
5. **Versioned Datasets:** Track historical data versions to identify when shifts begin and roll back if necessary.

Data Leakage

- **Data leakage** happens when information that **should not be available at prediction time** is used to train the model.
- Data leakage occurs when the model learns from future or outside information that it wouldn't have access to in real use.
- Result:
 - Unrealistically **high training/validation performance**
 - **Poor real-world performance**
 - False confidence in the model

Why Data Leakage is Dangerous

- Validation scores lie to you
- Model fails in production
- Wastes time, money, and trust
- Very hard to detect after deployment

Main Types of Data Leakage

1. Target Leakage (Most Dangerous):

When features contain **direct or indirect information about the target**.

Example:

- Predicting *loan default*
- Feature: `loan_status = "approved / rejected"`
- That's basically the answer!

More subtle examples:

- Using `final_grade` to predict pass/fail
- Using `total_amount_paid` to predict `will_default`

2. Train-Test Contamination

When information from the **test set leaks into training**.

Common cases:

- Scaling or normalization done on **entire dataset before split**
- Feature selection using full data
- Imputation using global mean (train + test)

3. Temporal Leakage (Very Common in Real Systems)

When **future data** is used to predict the past.

Example:

- Predicting stock price for day t
- Using moving average calculated with data from $t+1$

Especially dangerous in:

- Time series
- Fraud detection
- Recommendation systems

4. Duplicate / Group Leakage

Same entity appears in both train and test.

Example:

- Same customer transactions in both sets
- Same patient records split randomly

Solution:

- Use **GroupKFold** or split by customer_id, patient_id

How to Detect Data Leakage

Red Flags

- Validation accuracy **too good to be true**
- Huge drop from validation → production
- Model performs well with very few features
- Feature importance dominated by 1–2 features

Practical Checks

- Ask: “*Would this feature exist at prediction time?*”
- Shuffle target → model should perform badly
- Train with only top feature → suspiciously high accuracy = leakage

Prevention Methods

1. Split Data First

Always split **before**:

- Scaling
- Imputation
- Encoding
- Feature selection

2. Use Pipelines

Pipelines prevent leakage automatically.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

3. Time-Aware Splitting

For time-series data:

- Train on past
- Test on future

Handling Imbalanced Data for Classification

Imbalanced data occurs when one class has far more samples than others, causing models to favour the majority class and perform poorly on the minority class. This often results in misleading accuracy, especially in critical applications like fraud detection or medical diagnosis.

- ML models tend to get biased toward the majority class and predict it more frequently.
- Minority class instances may be treated as noise, causing the model to overlook them.
- Accuracy becomes misleading because the model performs well only on the dominant class.
- Skewed decision boundaries lead to poor generalisation and weak performance on minority class predictions.

Techniques to Handle Imbalanced Data

1. Use Better Evaluation Metrics

Accuracy is not a reliable metric for imbalanced datasets because a model can predict only the majority class and still achieve high accuracy. Instead, metrics like Precision, Recall and F1-score give a clearer picture of performance.

- Precision measures how many predicted positives are actually correct.
- Recall measures how many actual positives the model correctly identifies.
- F1-score is the harmonic mean of precision and recall and is widely used for imbalanced datasets.

Precision and F1 score decrease with more false positives while recall and F1 score drop when minority class samples are missed. F1 score improves only when both precision and recall increase making it ideal for imbalanced datasets.

2. Resampling Techniques

Resampling adjusts the size of classes to make them more balanced.

- **Over sampling:** Duplicates or generates minority class samples to help the model learn more patterns.
- **Under sampling:** Removes majority class samples to balance the dataset and give equal importance to all classes.

```

import numpy as np
from sklearn.datasets import make_classification
from imblearn.over_sampling import RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler
from collections import Counter

X, y = make_classification(n_classes=2, class_sep=2, weights=[0.1, 0.9],
                          n_informative=3, n_redundant=1, flip_y=0,
                          n_features=20, n_clusters_per_class=1,
                          n_samples=1000, random_state=42)

print("Original class distribution:", Counter(y))

oversample = RandomOverSampler(sampling_strategy='minority')
X_over, y_over = oversample.fit_resample(X, y)
print("Oversampled class distribution:", Counter(y_over))

undersample = RandomUnderSampler(sampling_strategy='majority')
X_under, y_under = undersample.fit_resample(X, y)
print("Undersampled class distribution:", Counter(y_under))

```

Output:

```

Original class distribution: Counter({np.int64(1): 900, np.int64(0): 100})
Oversampled class distribution: Counter({np.int64(1): 900, np.int64(0): 900})
Undersampled class distribution: Counter({np.int64(0): 100, np.int64(1): 100})

```

3. Balanced Bagging Classifier

- Balanced Bagging Classifier is an ensemble technique used to handle imbalanced datasets.
- It works like Bagging but balances each bootstrap sample so that minority classes are not ignored during training.
- Standard models favour the majority class due to which minority class predictions become poor. Balanced Bagging Classifier resamples data internally to fix this problem.

4. Synthetic Minority Oversampling Technique (SMOTE)

- SMOTE is a data level technique used to handle imbalanced datasets by creating new synthetic samples for the minority class instead of duplicating existing ones.
- In SMOTE a minority class data point is first selected then its k-nearest neighbours are identified and finally a new synthetic data point is generated between the selected point and one of its neighbours.

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from collections import Counter
X, y = make_classification(
    n_classes=2,
    class_sep=2,
    weights=[0.1, 0.9],
    n_informative=3,
    n_redundant=1,
    flip_y=0,
    n_features=20,
    n_clusters_per_class=1,
    n_samples=1000,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Class distribution before SMOTE:", Counter(y_train))
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
print("Class distribution after SMOTE:", Counter(y_train_resampled))

```

Output:

```

Class distribution before SMOTE: Counter({np.int64(1): 713, np.int64(0): 87})
Class distribution after SMOTE: Counter({np.int64(1): 713, np.int64(0): 713})

```

5. Threshold Moving

In classification, models often give probabilities for each class. By default, a probability above 0.5 is labelled as the positive class. However, for imbalanced dataset this default may not work well.

To improve performance the classification threshold can be adjusted. A lower or higher threshold can help the model better detect the minority class or reduce false positives.

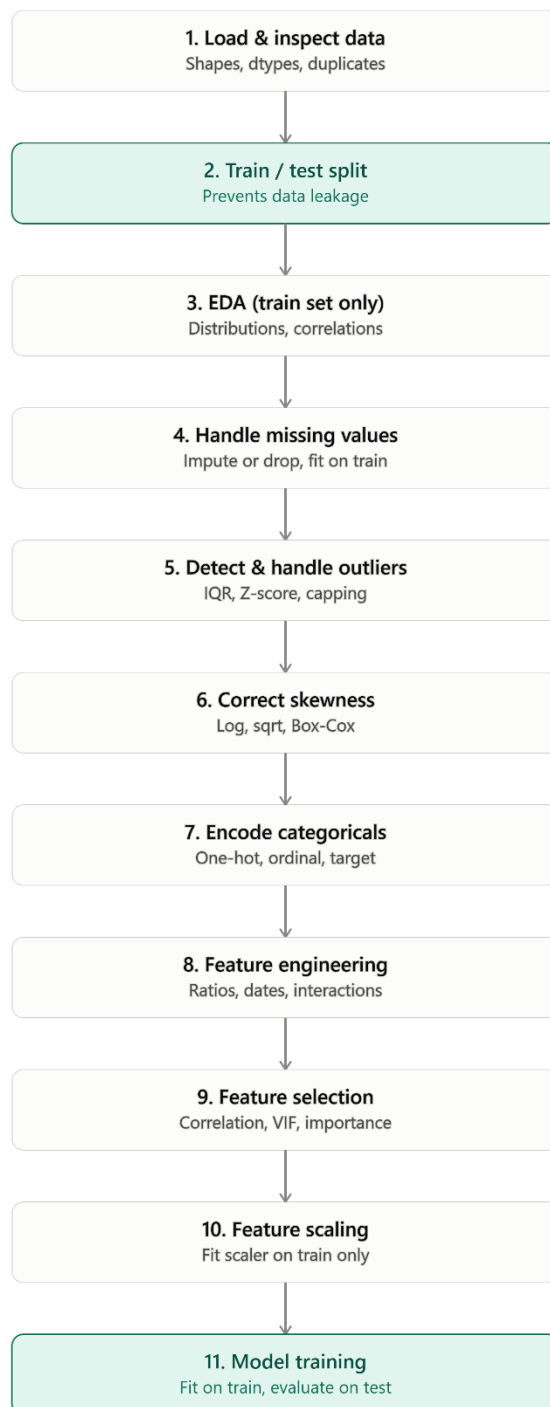
Ways to find the best threshold:

- **ROC Curve:** Choose the threshold that balances true positive rate and false positive rate.
- **Precision-Recall Curve:** Especially useful for imbalanced data to maximize precision and recall.
- **Grid Search / Range Testing:** Experiment with multiple threshold values to find the one that gives the best performance metrics.

Evaluation Metrics for ML Models

Regression ML Models

Steps To Solve ML Problem



Recommended sequence

1. Data extraction and reading

Load data, check shapes, data types, basic sanity (duplicate rows, ID uniqueness).

2. Train-test split (do this early!)

Right after loading, before EDA, outlier handling, imputation, scaling, encoding — split into train/test (or train/val/test). This is the step often missing from these lists, and skipping it is the #1 cause of leakage. Every statistic used later (mean for imputation, IQR for outliers, scaler parameters, encoding maps) should be *learned on train only* and *applied* to test.

3. EDA

Explore the **training set** only: distributions, correlations, target relationship, missingness patterns, class balance, cardinality of categorical. This informs every decision downstream (what to impute with, what to transform, what to encode how).

4. Handling null values

Do this before outlier detection and skew correction, because:

- Outlier detection methods (Z-score, IQR) get distorted by NaNs.
- Skewness/transformations can't be computed with missing values present.
Decide: drop vs impute (mean/median/mode, KNN, model-based), and consider adding "was_missing" indicator flags if missingness is informative.

5. Outlier detection and handling

Now that nulls are handled, detect outliers (IQR, Z-score, isolation forest, domain caps). Decide: cap/winsorize, transform, or remove — removing is risky on small data or if outliers are genuine signal.

6. Handling skewness

Apply after outliers are addressed, since extreme outliers heavily distort skew and the transformation choice (log, sqrt, Box-Cox, Yeo-Johnson). Do this mainly for numeric features going into linear/distance-based models; tree-based models don't need it.

7. Encoding categorical variables

One-hot, label, target/mean encoding, ordinal, etc. Target encoding especially must be fit on train and mapped onto test (or done with cross-fold encoding) to avoid leakage.

8. Feature engineering

Create new features (ratios, date parts, interactions, aggregations) — do this before final selection so the selection process can evaluate the new features too.

9. Feature selection

Now that all features are numeric, cleaned, and engineered, apply selection (correlation filtering, VIF, mutual information, model-based importance, RFE).

10. Feature scaling

Do this **last**, right before modelling and only fit the scaler on train, transform both train/test with those parameters. Scaling early and then doing more feature engineering means you'd need to rescale anyway, so it's cleanest at the end. (Tree-based models don't need this step at all.)

Key principles regardless of order

- **Split before you compute any statistic** you'll reuse (mean, std, IQR bounds, category frequencies, scaler params) — fit only on train, transform both.

- **EDA drives decisions**, not the reverse — look at your data before choosing imputation/outlier/transform strategies.
- **Model type matters**: tree-based models (RF, XGBoost) are largely immune to scaling and skewness, so steps 6 and 10 can be skipped for them; linear/SVM/KNN/neural nets need them.
- **Iterate**: this isn't strictly linear in practice — EDA findings from step 3 often send you back to reconsider step 4/5 choices, and feature engineering can surface new outliers/nulls needing another pass.